

Lo primero no fue un bucket. Fue la **alerta de gasto**.

Terraform levantó dieciocho recursos. Ninguno factura por hora, así que la cuenta puede quedarse encendida indefinidamente y seguir costando cero. Pero el orden en que se crean importa más que la lista.

Dos módulos, y el huevo y la gallina

El módulo que crea el bucket donde vivirá el estado no puede guardar su propio estado dentro de él. Esa restricción define toda la estructura.



El bucket de estado lleva `prevent_destroy`: es el único recurso que sobrevive al `terraform destroy` final. Sin el estado, todo lo demás queda huérfano y deja de poder gestionarse con Terraform.

Los tres frenos de coste

No son optimizaciones. Son topes que impiden que un descuido se convierta en una factura.

Presupuesto antes que infraestructura

5 USD al mes con aviso al 50%, al 100% y al 100% **previsto**. Este último proyecta el ritmo de gasto y avisa antes de llegar, que es cuando todavía hay margen de reacción.

Tope de escaneo en Athena

Athena factura por terabyte leído. Una consulta mal escrita sobre una tabla sin particionar puede costar dinero de verdad. El workgroup aborta la que supere 1 GiB.

Sin crawler de Glue

Un crawler en *schedule* factura DPU cada vez que corre, haya novedades o no. Es la forma más común de gastar en Glue sin enterarse. La tabla la registrará el propio job de PySpark.

Las decisiones de esta fase

01 Bloqueo de estado sin DynamoDB

Terraform 1.10 introdujo `use_lockfile` para el backend de S3. Antes hacía falta una tabla de DynamoDB dedicada solo a eso. Un recurso menos que crear, documentar y pagar.

02 Ni un ID de cuenta escrito a mano

El repositorio es público. El bucket de estado usa un sufijo aleatorio; el resto obtiene el ID con `data.aws_caller_identity` en tiempo de ejecución.

03 IAM sin políticas gestionadas

Ni un `AmazonS3FullAccess`. La Lambda solo puede escribir en `raw` —no leer, no borrar— y el job de Glue opera únicamente sobre su base de datos del catálogo.

04 Los logs se declaran, no se heredan

Si Lambda crea su grupo de logs solo, nace **sin retención**: se guarda para siempre y se paga para siempre. Declarado en Terraform, caduca a los 14 días.

Lo que falló

- Una coma tumbó el despliegue

`InvalidTag: The TagValue you have provided is invalid`

Falló en **raw** y **curated**, pero no en el bucket de resultados. La diferencia estaba en el valor de una etiqueta: los dos primeros llevaban una coma.

Los valores de etiqueta de un bucket S3 aceptan un juego de caracteres más restringido que el del resto de servicios de AWS. **La misma etiqueta funciona en un rol IAM y revienta en S3.** Y el mensaje de error no dice cuál es el carácter problemático, ni siquiera qué etiqueta lo contiene.

El *apply* quedó a medias. Al corregir y volver a planificar, Terraform pasó de 23 recursos a 18: ya tenía registrado lo creado. Es exactamente para lo que sirve el estado, y se agradece verlo funcionar cuando algo se rompe.

Qué cuesta cada cosa

Verificado con la CLI sobre los recursos reales, no sobre el estado local.

RECURSO	Nº	COSTE MENSUAL
Buckets S3 (vacíos)	4	~0,00 \$ · 0,023 \$/GB cuando tengan datos
Roles IAM con políticas inline	2	0,00 \$ · IAM siempre es gratis
Base de datos del catálogo	1	0,00 \$ · gratis hasta 1M de objetos
Workgroup de Athena	1	0,00 \$ · se paga por consulta
Grupo de logs	1	~0,00 \$ · 0,50 \$/GB al escribir
Presupuesto	1	0,00 \$ · los dos primeros son gratis

Caudalit · Fase 3 de 10 · Infraestructura

Siguiente: ingesta con Lambda, reintentos contra una API que falla sola